Rutgers ECE 434 Project 2 Report

Names: Names: Samarth Sathishkanth (ss4714), Doug Lavin(dml336), Daniel Kang (dk1203), Aryan Kini (apk84)

Group number: 25

1 Project 1 Code Structure

The original Project 1 program is still in `project1.c`. It builds a recursive process tree with `fork()`. Each process owns a slice of the generated input array. If the slice is still large and the process limit allows more processes, it splits the slice into `branch` children.

Hidden nodes are represented as negative integers in the array. `compute_local()` treats every negative value as a hidden key and stores its index in `Result.hidden_pos[]`. The number of hidden nodes found is `Result.hidden_found`.

Child-to-parent communication already used pipes. Each child writes one binary `Result` struct to its parent. The parent uses `poll()` to wait for child pipe data, reads each `Result`, merges metrics, then uses `waitpid()` to analyze child termination.

2 Problem 1 Design

I kept `project1.c` as the Project 1 baseline and wrote the Project 2 signal version in `problem1_signals.c`. The new program keeps the same basic design: generated input array, recursive segment processing, one pipe per child, and `waitpid()` in every parent.

For the Project 2 run target I used:

```
./problem1_signals exp1 fast
./problem1_signals exp2 fast
```

Fast mode keeps the same behavior but shortens sleeps so the output files can be generated. The default mode uses the longer assignment-style sleeps: Rule 1 sleeps 100 seconds, Rule 3 parent waits 10 seconds, and Rule 3 child sleeps 30 seconds.

After a child computes its metrics, it writes its `Result` through the pipe and then calls `raise(SIGTSTP)`. In the redirected `make` environment, `SIGTSTP` can be discarded because the process group is not an interactive job-control process group. To make the test run observable, the child then raises `SIGSTOP` as a compatibility stop. The required `SIGTSTP` call is still present, and the parent resumes children with `SIGCONT`.

The parent reads all sibling hidden counts first. In the generated test case the three direct children had:

```
parent=10 siblings:
  child=11 hidden=12 child=12 hidden=30 child=13 hidden=108
```

The parent updates its own hidden-node list using only the child with the fewest hidden nodes. This is why the final propagated hidden-node count is 12 even though all children found more hidden nodes in total.

3 Problem 1 Observations

The stopped children were detected with `waitpid()` using `WUNTRACED` and `WNOHANG`:

```
[waitpid] pid=11 raw_status=4991 WIFSTOPPED yes, WSTOPSIG=19 (SIGSTOP)
[waitpid] pid=12 raw_status=4991 WIFSTOPPED yes, WSTOPSIG=19 (SIGSTOP)
[waitpid] pid=13 raw_status=4991 WIFSTOPPED yes, WSTOPSIG=19 (SIGSTOP)
```

Rule 1 was applied to the highest hidden-node child:

```
rule=1 parent=10 child=13 hidden=108
rule=1 child=13 exit=14
[waitpid] pid=13 raw_status=3584 WIFEXITED yes, WEXITSTATUS=14
```

The return value from `waitpid()` was the child PID. The status value was decoded with `WIFEXITED` and `WEXITSTATUS`. This child exited normally with its unique exit argument.

Rule 2 was applied to the middle child. The parent sent `SIGUSR1` using `sigqueue()` and passed secret signal number `SIGTERM`. The child handled the secret signal, restored the default action, then raised it again so termination showed as signal termination:

```
rule=2 parent=10 child=12 hidden=30
handler sig=10(SIGUSR1) pid=12 ppid=10 sender=10 secret=15(SIGTERM)
secret handler pid=12 sig=15(SIGTERM)
[waitpid] pid=12 raw_status=15 WIFSIGNALED yes, WTERMSIG=15 (SIGTERM)
```

Here `waitpid()` again returned the child PID, but the status was decoded with `WIFSIGNALED` and `WTERMSIG`. The terminating signal was `SIGTERM`.

To observe what happens to offspring, the Rule 2 child forks a helper child and then terminates without waiting for it. The helper output showed adoption by PID 1:

```
orphan helper pid=14 ppid=12 start
orphan helper pid=14 ppid=1 after parent exit
```

The child process's offspring are not automatically killed just because their parent dies. They become orphans and are adopted by init/systemd. Direct children are still waited for by their own parent, so the main program does not leave zombies.

Rule 3 was applied to the lowest hidden-node child. In Experiment 1, `SIGINT` had a handler that printed and returned:

```
rule=3 parent=10 child=11 hidden=12
SIGINT handler sig=2(SIGINT) child=11 ppid=10 sender=10
rule=3 child=11 sleep_left=3 sigint_count=1
rule=3 child=11 wait SIGQUIT
[waitpid] pid=11 raw_status=131 WIFSIGNALED yes, WTERMSIG=3 (SIGQUIT)
```

This shows that when a caught signal interrupts `sleep()`, `sleep()` returns early with remaining time, and the child moves to the next instruction after the handler returns.

In Experiment 2, `SIGINT` was ignored. There was no `SIGINT` handler output. The child stayed in sleep until the parent sent `SIGQUIT`, and `waitpid()` reported signal termination by `SIGQUIT`.

4 Problem 2 Design

`problem2_signals.c` has three modes:

```
./problem2_signals part1
./problem2_signals q2
./problem2_signals q3
```

The Makefile run targets use `fast` mode for output generation. In normal `part1`, the child loop uses the assignment's 10-second sleep per iteration. In fast mode, it computes the same sum without the long per-iteration delay.

Part 1 forks four child processes. The parent initially ignores the eight listed signals while forking:

```
SIGINT, SIGABRT, SIGILL, SIGCHLD, SIGSEGV, SIGFPE, SIGHUP, SIGTSTP
```

After forking, the parent installs `SA_SIGINFO` handlers for the same signals. Each child catches four signals, blocks two other signals for its whole execution, and ignores the rest. The two blocked signals are not caught; they keep the default disposition but stay blocked so they can appear in the pending queue. The handler prints signal number, signal name, recipient PID, parent PID, and sender PID.

Example child setup from `outputs/problem2_part1.txt`:

```
Child index=1 pid=12 configured signal policy
  catches: SIGSEGV SIGFPE SIGHUP SIGTSTP
  blocked for full execution: SIGINT SIGABRT
```

5 Problem 2 Part 2 Answers

When the parent receives a signal externally, the result depends on its current disposition:

Signal	While parent ignores before forks	While parent catches during child run	After restoring defaults
`SIGINT`	discarded	handler prints and parent continues	terminates parent
`SIGABRT`	discarded	handler prints and parent continues	aborts, usually core dump
`SIGILL`	discarded	handler prints and parent continues	terminates, usually core dump
`SIGCHLD`	discarded/ignored	handler prints; also happens when children exit	usually ignored if no unwaited child
`SIGSEGV`	discarded	handler prints and parent continues	terminates, usually core dump
`SIGFPE`	discarded	handler prints and parent continues	terminates, usually core dump
`SIGHUP`	discarded	handler prints and parent continues	terminates parent
`SIGTSTP`	discarded	handler prints and parent continues	stops parent

If a signal is ignored, sending it more than once makes no visible difference. If it is caught and unblocked, the handler can run for each delivery. If it is blocked, standard signals are not queued as multiple copies; several sends normally leave one pending instance. If the default action terminates the process, the first terminating signal ends the program, so later sends do not matter.

6 Problem 2 Q2 Answers

In Q2, child 0 sends every listed signal twice to child 1 and twice to the parent.

For the parent, the signals are caught, so the parent prints handler output and continues:

```
[handler] signal=11 (SIGSEGV), recipient pid=10, parent pid=9, sender pid=11
[handler] signal=20 (SIGTSTP), recipient pid=10, parent pid=9, sender pid=11
```

For child 1, the result depends on child 1's mask and dispositions. Child 1 catches `SIGSEGV`, `SIGFPE`, `SIGHUP`, and `SIGTSTP`; blocks `SIGINT` and `SIGABRT`; and ignores the other listed signals. The pending queue before it exited showed:

```
Q2 receiver child before exit pid=12 pending: SIGINT SIGABRT
```

Even though each was sent twice, only one pending `SIGINT` and one pending `SIGABRT` appeared. That is because these are standard signals, not real-time signals, so repeated blocked deliveries coalesce.

7 Problem 2 Q3 Answers

The Q3 code includes the eight assignment signals plus `SIGQUIT`, because Q3 adds `SIGQUIT` to the masking experiment. The Linux/POSIX call is named `sigtimedwait`, even though the assignment text says `sigtimewaitinfo`.

The parent blocks `SIGINT`, `SIGQUIT`, and `SIGTSTP` before forking. It sends itself all Q3 signals three times before forking. The parent pending queue before fork was:

```
Q3 parent pending before fork pid=10 pending: SIGINT SIGQUIT SIGTSTP
```

The non-blocked signals were handled immediately. The blocked signals were pending. Multiple sends of the same standard signal still produced one pending signal.

The children inherited the parent's handlers after fork. They did not inherit the parent's pending queue. After the parent sent signals to every child, the first half of children blocked the same three-signal set and showed the same pending queue:

```
Q3 child pending before sigwait calls pid=11 pending: SIGINT SIGQUIT SIGTSTP
Q3 child pending before sigwait calls pid=12 pending: SIGINT SIGQUIT SIGTSTP
```

The other half blocked the remaining six signals and showed:

```
Q3 child pending before sigwait calls pid=13 pending:
  SIGABRT SIGILL SIGCHLD SIGSEGV SIGFPE SIGHUP
Q3 child pending before sigwait calls pid=14 pending:
  SIGABRT SIGILL SIGCHLD SIGSEGV SIGFPE SIGHUP
```

Then `sigwait`, `sigwaitinfo`, and `sigtimedwait` consumed pending signals from the blocked sets. For example:

```
Q3 parent sigwait consumed 2 (SIGINT) in pid=10
Q3 parent sigwaitinfo consumed 3 (SIGQUIT) in pid=10 from sender=10
Q3 child sigwait consumed 4 (SIGILL) in pid=13
Q3 child sigwaitinfo consumed 8 (SIGFPE) in pid=13 from sender=10
```

Processes that block the same signals have the same type of pending queue after receiving the same signals. The child queues are not copies of the parent's queue from before fork; they are built from signals sent directly to those child PIDs after fork. Non-blocked signals run the inherited handlers immediately.

8 How To Compile And Run

```
make clean
make
```

```
make run-problem1
make run-problem1-exp1
make run-problem1-exp2
make run-problem2-part1
make run-problem2-q2
make run-problem2-q3
```

Generated output files:

```
outputs/problem1_exp1.txt
outputs/problem1_exp2.txt
outputs/problem1_default.txt
outputs/problem2_part1.txt
outputs/problem2_q2.txt
outputs/problem2_q3.txt
```

9 Known Limitations

The run targets use fast mode so grading/test output completes quickly. Running without `fast` gives the longer sleeps required by the assignment.

The `SIGTSTP` behavior is affected by whether the program is run as an interactive job. In this redirected Makefile environment, I added `SIGSTOP` after the required `raise(SIGTSTP)` so the stopped-child behavior is still observable.

Because several processes write to the same output file, some ordinary progress lines may interleave. The important signal, pending queue, and wait-status lines are printed clearly enough to support the observations above.